

CHAPTER NO.	TITLE	PAGE NO.
1	ABSTRACT	1
2	INTRODUCTION	2
3	HISTORY	3
4	HARDWARE DESCRIPTION	4
5	DESIGN AND IMPLEMENTATION	10
6	SOFTWARE DESCRIPTION	12
7	SOFTWARE CODING	13
8	WORKING	17
9	APPLICATION	20
10	CONCLUSION	23

1.ABSTRACT

The application of radio detection and ranging in different places such as military installation, commercial use is done with the Help of RADAR SYSTEM which uses electromagnetic waves for detection of different physical components such as distance, speed, position, Range, direction, size etc., which can be either fixed or be in motion. Use of radar system has been developed greatly specially in field of navigation. In this research we study about existing navigation Technologies and proposed an Arduino based radar system. It has advantage over other radar system as kit reduces power consumption and Connect programmer to wide range or Arduino programmers and opensource code. The system consist a basic ultrasonic sensor placed upon A servo motor which rotates at a certain angle and speed. This ultrasonic sensor is connected to Arduino digital input output pins and servo Motor also connected to digital input output pins.

KEYWORDS :Arduino, ultra-sonic, radar, positioning, surveillance, obstacle detection.

2. INTRODUCTION

We know everything produces sound wave just by existence and effect flow of air around them with their natural frequency. These Frequencies are beyond hearing range of humans. Wave of frequency range of 20000hz and thereabouts are called ultra-sonic wave and these Waves can be detected by an ultrasonic sensor which helps us to get various knowledge. An Ultrasonic detector usually has a transducer which convert sound energy into electrical energy and electrical energy into sound energy. They are used for measuring object position and orientation, collision avoidance system, surveillance system etc.

Ultrasonic technology provide relief from problem such as linear measurement problem, as it allows user to get non-contact measurements in this Way distance between object and its speed etc can me easily measured. Speed of travel of sound wave depends upon square root of ratio between medium density and stiffness. Also, property of speed of sound can Also be changed by natural environment condition like temperature. So , basically, an ultrasonic sensor sends ultrasonic waves which travels in air and gets reflected after striking any object. By studying the Property of reflected wave, we can get knowledge about objects distance, position, speed etc.

A processing software and an Arduino software is used with hardware system for detection of objects various parameters. One of the most common application of ultra-sonic sensor is range finding. It is also called as sonar which is same as radar in which ultrasonic Sound is directed at a particular direction and if there is any object in its path it strikes it and gets reflected back and after calculation time taken to come back we can determine distance of object. In real life this method is used by bats.

The block diagram of **Arduino Ultrasonic Radar** is shown in figure 1.

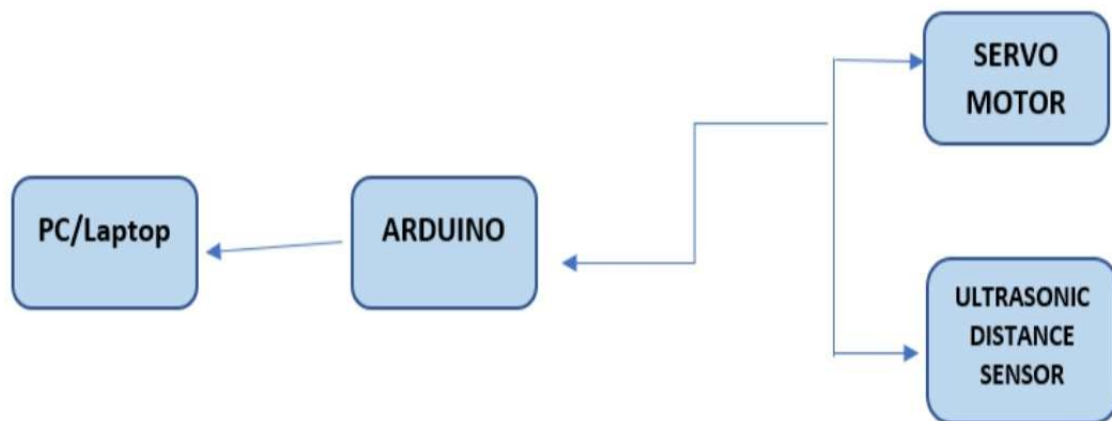


Fig. 1. Block diagram of Arduino Ultrasonic Radar

3. History

It is a common assumption that radar is a by-product of World War II, brought about by military necessity. According to Hiser (Radar Meteorology, Third Edition, [1970](#)), this is true only to the extent that the war produced radar equipment in great quantities suitable for practical use. Actually, the fundamental principle underlying all radars was first observed in [1886](#) by the physicist Heinrich Hertz when he found that electromagnetic waves could be reflected from various objects, and even focused into beams by appropriate reflectors.

In [1904](#), a German engineer, Hulsmeyer, was granted a patent in several countries on a proposed method of using electromagnetic waves in an obstacle detector and navigation aid for ships. In [1922](#), Marconi urged the use of "short waves" for radio detection. From these beginnings, radar gradually evolved; it was not a science which was suddenly discovered. Various experimenters in electromagnetic waves during the following years reflected these waves from the upper atmosphere, and verified the existence of a series of ionized layers known as the ionosphere

3. HARDWARE DESCRIPTION

Ultrasonic sensor



Fig. 2. A type of ultrasonic sensor

The HC-SR04 ultrasonic sensor uses sonar to determine distance to an object like bats or dolphins do. It offers non-contact range detection from 2cm to 400 cm or 1" to 13 feet. Its operation is not affected by sun light or black material like sharp rangefinders are (although acoustically soft materials like cloth can be difficult to detect). It comes complete with ultrasonic transmitter and receiver module. The human ear can only detect sounds between 2012-20kHz. The sound waves beyond 20kHz are called ultrasonic waves or ultrasound. The principle of ultrasonic distance measurement uses the velocity of ultrasonic waves spreading in air, measuring the time from launch to reflection when it encounters an obstacle. We then calculate the distance between the transmitter and the obstacle according to the time and the velocity. Thus, the principle of ultrasonic distance measurement is the same as with a radar.

Features

Electrical Parameters	HC-SR04 Ultrasonic module
Operating Voltages	(DC)5V (4.5-5.5V)
Quiescent Current(inactivity)	2mA (1.5-2.5 mA)
Working Current	15mA(10-20 mA)
Operating Frequency	40KHz (Ultrasonic)
Farthest range	4m
Nearest range	2cm
Resolution	0.3cm
Measuring angle	15 degrees
Input trigger signal	10us TTL pulse
Output echo signal	Output TTL level signal, proportional with range

Dimensions	45*20*15 mm
------------	-------------

Since the sensor considers the distance to an obstacle that reflects the ultrasonic wave, we cannot use it to make trustworthy measurements of the position of moving objects. This is majorly due to the fact that this sensor works on the principle of transmitting a set of pulses and receiving their reflection, before transmitting again, as we shall see later.

Hence, there is no way that we can continuously track the position of an object with such a sensor. As far as we know, that can only be done by some type of a camera, which would've introduced the concepts of computer vision and taken us beyond the scope and costs of the current project

SERVO MOTOR

A servomotor is a rotary actuator that allows for precise control of angular position, velocity and acceleration. It consists of a suitable motor Coupled to a sensor for position feedback. It also requires a relatively sophisticated controller, often a dedicated module designed specifically for Use with servomotors. Servomotors are not a different class of motor, on the basis of fundamental operating principle, but uses servomechanism To achieve closed loop control with a generic open loop motor. Servomotors are used in applications such as robotics, CNC machinery or Automated manufacturing. A type of servo motor is shown in figure 3.



Fig. 3. A type of Servo motor

ARDUINO UNO

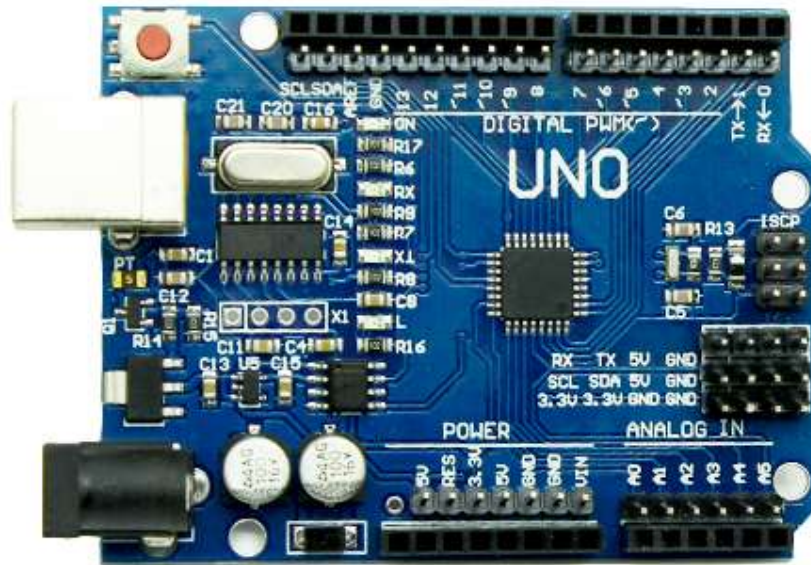


Fig. 4. Arduino Board

Arduino is an open source electronics platform for fast prototyping of projects for users with minimal knowledge or experience in electronics and programming. We have used the Arduino UNO, the most widely used variant of the Arduino.

Technically, Arduino Uno is a microcontroller board based on the 8-bit ATmega328P microcontroller. With 14 digital input/output pins (including 6 PWM outputs), 6 Analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button, it has everything needed to support the microcontroller, we simply have to connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. Arduino also comes with the Arduino IDE, in which we can write code and upload to the

Arduino. The programming language used for an Arduino, called the Arduino Programming language, is very similar to C/C++ except that we can use inbuilt functions of the Arduino libraries which keep the code very simple and short

Why Arduino?

We had the following reasons strongly backing our decision to choose the Arduino UNO Support Thanks to its simple and accessible user experience,

Arduino has been used in thousands of different projects and applications. The Arduino software is easy-to-use for beginners, yet flexible enough for advanced users. It runs on Mac, Windows, and Linux.

This has enabled a very large support base for Arduino, more than any other microcontroller board ever released in the market. Every problem/bug/ question that we ever had throughout the course of this project was promptly answered by questions in online forums, as there was a very high

probability that scores of people had already posted the same question and the issue was resolved by other experienced users.

Microcontroller	ATmega328P(8-bit)
Operating Voltage	(DC)5V
Input Voltage	(recommended) 7-12V; (limits) 6-20V
Digital I/O Pins	14(of which 6 provide PWM output)
Analog Input Pins	6
DC Current per I/O Pin	40mA
DC Current for 3.3V pin	50mA
Flash Memory	32 KB (ATmega328) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328)
EEPROM	1 KB (ATmega328)
Clock Speed	16MHz
Need of External Programmer	NO
Built-in-LED	pin 13
Length	68.6 mm
Width	53.4mm
Weight	25 g

Table 1.1: Features of Arduino UNO

Cost Effective - The Arduino UNO, its basic variant that every beginner to embedded systems usually starts with, is available for less than \$25. Being an open source hardware and due to its simplicity, it is easily and widely replicable, which is also responsible for the growth of such a large community as mentioned in the earlier reason.

Features - It is generally observed that the computing power and inbuilt memory of the Arduino easily supports a wide variety of circuits with multiple components along with continuous serial communication with another device/computer. Compared to the system configuration of the board, our project would require fairly low amount of resources.

The no. of GPIO pins on the UNO is more than sufficient considering the need for the circuitry in our project, as we shall see when we look at the pinouts of the servo motor and the sensor the only two other devices being used in the project.

One important advantage of using the UNO is that unlike most previous programmable circuit boards, the Arduino UNO does not need a separate piece of hardware (called a programmer) in order to load new code onto the board we can simply use a USB cable. The ATmega328 on the Arduino Uno comes pre-programmed with a bootloader that allows us to upload new code to it without the use of an external hardware programmer.

Moreover, the components we have used work under the same operating voltage and current conditions as an Arduino, especially considering the fact that such components are nowadays sold with their compatibility with Arduino in mind.

Considering the above factors, we found Arduino UNO to be the most appropriate microcontroller board to work with for this project.

Operation of Arduino

Once a program written for an Arduino, called a "Sketch is uploaded to the board via the Arduino IDE, the Arduino environment performs some small transformations to make sure that the code is correct C/C++. It then gets passed to a compiler, which turns the human readable code into machine readable instructions (object files). Then, the code gets combined with (linked against) the standard Arduino libraries that provide basic functions like digitalWrite() or Serial.print(). The result is a single

Intel hex file, which contains the specific bytes that need to be written to the program (flash) memory of the chip on the Arduino board. This file is then uploaded to the board: transmitted over the USB or serial connection via the bootloader already on the chip or with external programming hardware

BREAD BOARD

Breadboards are one of the most fundamental pieces when learning how to build circuits. In this tutorial, you will learn a little bit about What breadboards are, why they are called breadboards, and how to use one. Once you are done you should have a basic understanding of how Breadboards work and be able to build a basic circuit on a breadboard. Figure 4 shows the bread board.

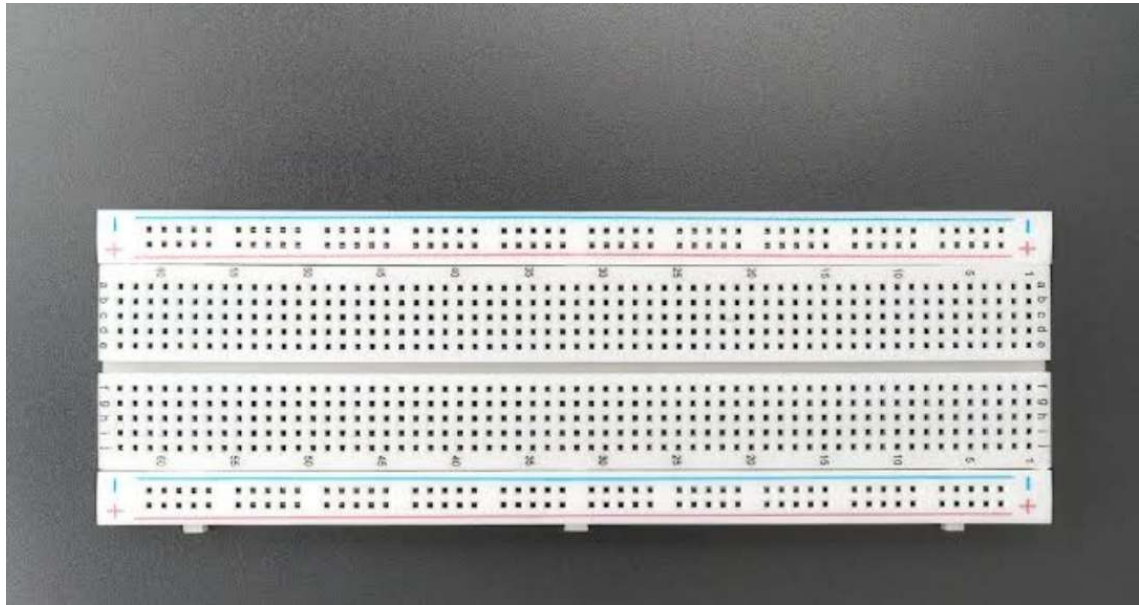


Fig. 4. Bread Board

Jumper wires

A jump wire (also known as jumper wire, or jumper) is an electrical wire, or group of them in a cable, with a connector or pin at each end (or sometimes without them simply "tinned"), which is normally used to interconnect the components of a breadboard or other prototype or test circuit, internally or with other equipment or components, without soldering.

Individual jump wires are fitted by inserting their "end connectors" into the slots provided in a breadboard, the header connector of a circuit board, or a piece of test equipment.

There are different types of jumper wires. Some have the same type of electrical connector at both ends, while others have different connectors

Types:



jumper wire

Some common connectors are:

Solid tips are used to connect on/with a breadboard or female header connector. The arrangement of the elements and ease of insertion on a breadboard allows increasing the mounting density of both components and jump wires without fear of short-circuits. The jump wires vary in size and colour to distinguish the different working signals.

Crocodile clips are used, among other applications, to temporarily bridge sensors, buttons and other elements of prototypes with components or equipment that have arbitrary connectors, wires, screw terminals, etc.

Banana connectors are commonly used on test equipment for DC and low- frequency AC signals.

Registered jack (RJnn) are commonly used in telephone (RJ11) and computer networking (RJ45).

RCA connectors are often used for audio, low-resolution composite video signals, or other low-frequency applications requiring a shielded cable.

RF connectors. are used to carry radio frequency signals between circuits, test equipment, and antennas.

RF jumper cables Jumper cables is a smaller and more bendable corrugated cable which is used to connect antennas and other components to network cabling. Jumpers are also used in base stations to connect antennas to radio units.

Usually the most bendable jumper cable diameter is 1/2".

5. DESIGN AND IMPLEMENTATION

CIRCUIT DESIGN

The circuit diagram of the Arduino radar sensor system shown in figure 5. The servo motor output is connected to 12th pin, Negative to ground and Positive to 5V. The TRIG and ECHO pins of the Ultrasonic sensor are connected respectively to pin 10 and 11 of the Arduino. Power supply with common GND is connected to the Servo Motor and Ultrasonic Sensor.

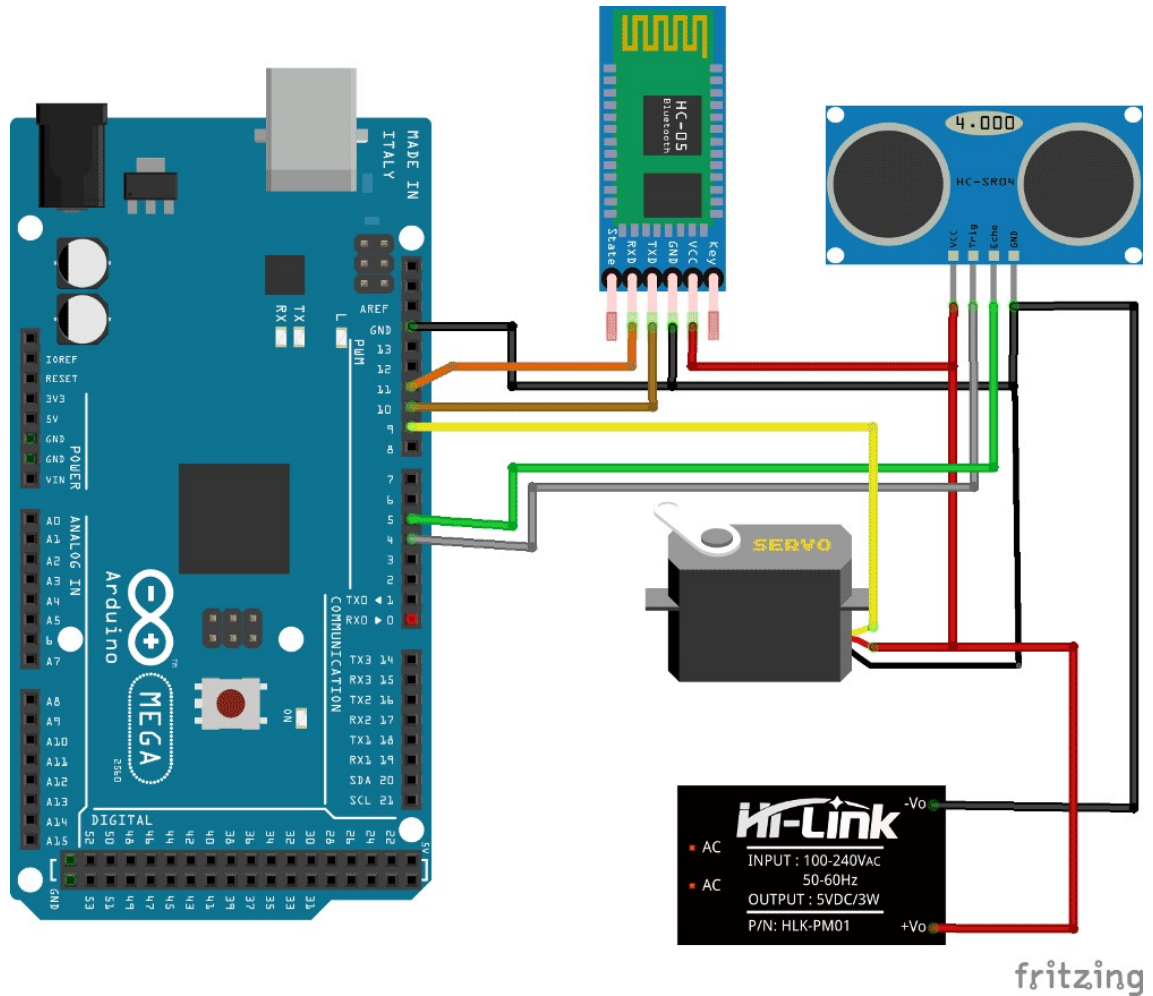


Fig. 5. Arduino radar sensor system

WORKING PROCEDURE

To start the Arduino Radar Sensor, you should know the programming code. There are two programming codes need to start the radar. One is the Arduino IDE and another one is the processing . After uploading the code to Arduino, the servo sweeps from 15° to 165° and back again to 15°.

Since the Ultrasonic Sensor is riding the Servo. Now open the processing application and run the code. If there is no error then another processing window open up. This is a Graphical representation of the data. The Ultrasonic Sensor represented in a radar type display. When the Ultrasonic Sensor detects any object within its range, the object will be shown on screen as a graphic image. A typical output is shown in figure 6.

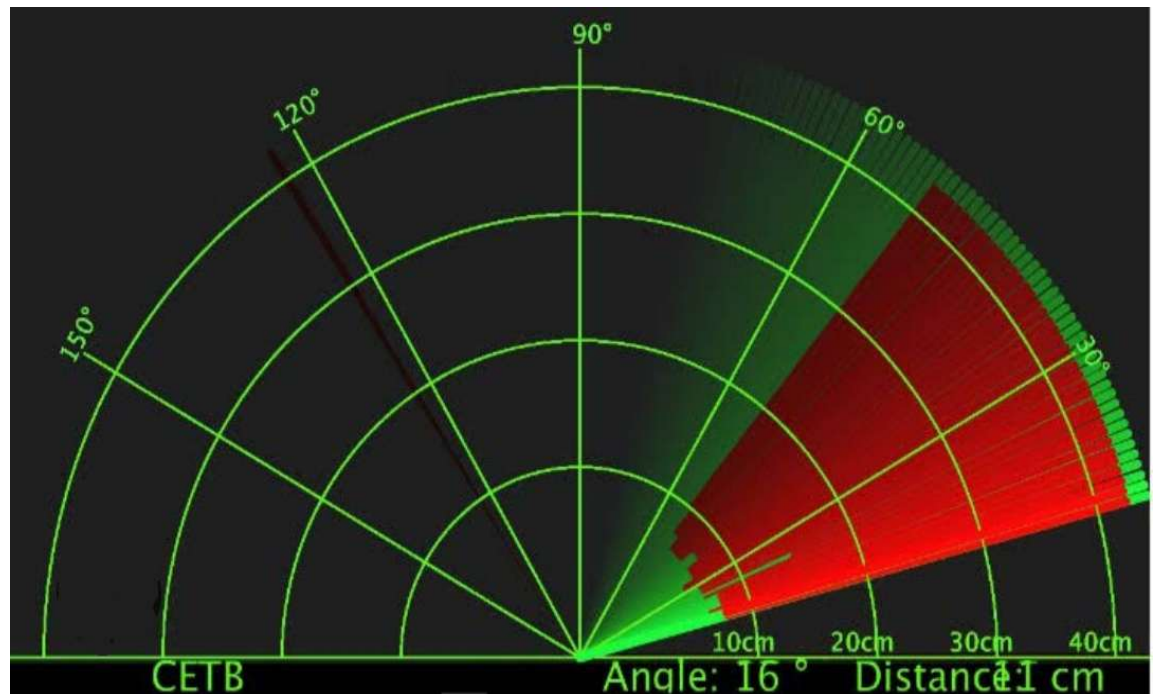


Fig. 6. Detected object

6. SOFTWARE DESCRIPTION:

ARDUINO IDE

The Arduino Integrated Development Environment (IDE) is a cross-platform application (for Windows, macOS, Linux) that is written in functions from C and C++. It is used to write and upload programs to Arduino compatible boards, but also, with the help of third-party cores, other vendor development boards. The Arduino IDE supports the languages C and C++ using special rules of code structuring.

PROCESSING

Processing is an open-source graphical library and integrated development environment (IDE) built for the electronic arts, new media art, and visual design communities with the purpose of teaching non-programmers the fundamentals of computer programming in a visual context . Processing uses the Java language, with additional simplifications such as additional classes and aliased mathematical functions and operations. It also provides a graphical user interface for simplifying the compilation and **execution stage**.

7. SOFTWARE CODING

Coding used to implement the project is given below.

```
// Includes the Servo library
#include <Servo.h>.

// Defines Trig and Echo pins of the Ultrasonic Sensor ConstinttrigPin = 10;
ConstintechoPin = 11;

// Variables for the duration and the distance

Long duration;

Int distance;

Servo myServo;

// Creates a servo object for controlling the servo motor

Void setup() {
pinMode(trigPin, OUTPUT);

// Sets the trigPin as an Output

pinMode(echoPin, INPUT);

// Sets the echoPin as an Input Serial.begin(9600); myServo.attach(12);

// Defines on which pin is the servo motor attached
}

Void loop()
{
// rotates the servo motor from 15 to 165 degrees For(inti=15;i<=165;i++){
myServo.write(i);

delay(30);

distance = calculateDistance();

// Calls a function for calculating the distance measured by the Ultrasonic sensor for each
degreemotoSerial.print(i);

// Sends the current degree into the Serial Port Serial.print(","); Serial.print(distance);

// Sends the distance value into the Serial Port

Serial.print(".");

// Sends addition character right next to the previous value needed later in the Processing IDE
for indexing

}

// Repeats the previous lines from 165 to 15 degrees For(inti=165;i>15;i--){
```

```

myServo.write(i);
delay(30);
distance = calculateDistance();
Serial.print(i);
Serial.print(",");
Serial.print(distance);
Serial.print(".");
}
} I
ntcalculateDistance()
{
digitalWrite(trigPin, LOW);
delayMicroseconds(2);
// Sets the trigPin on HIGH state for 10 micro seconds digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH);
// Reads the echoPin, returns the sound wave travel time in microseconds
distance= duration*0.034/2;
return distance;
}

```

5.1 Radar Systems with Arduino – Programming Language Details

[1. The Arduino Programming Language](#)

The Arduino programming language is a simplified version of C/C++, designed to be easy for beginners to use. It provides high-level abstractions for microcontroller programming, with built-in functions for hardware control like reading sensors, writing outputs, and handling communication.

2. Language Characteristics

Based on C/C++: You write code using standard C/C++ syntax.

Simplified API: Arduino provides easy-to-use functions like digitalWrite(), analogRead(), delay(), etc.

Real-time Execution: Code runs on the microcontroller in real time, perfect for control systems like radar.

3. Program Structure in Arduino

All Arduino programs follow this basic structure:

```
void setup() {  
  // Runs once when the Arduino is powered on or reset  
}  
void loop() {  
  // Repeats continuously until the Arduino is turned off  
}
```

For a radar system, you write code inside these two functions to control:

Sensors (like HC-SR04 for distance)

Motors (like SG90 for scanning)

Communication (like Serial Monitor for displaying data)

4. Example: Radar System Code Breakdown

```
#include <Servo.h> // Include library for servo motor  
const int trigPin = 9; // Ultrasonic Trigger Pin  
const int echoPin = 10; // Ultrasonic Echo Pin  
Servo servoMotor; // Create servo motor object  
void setup() {  
  Serial.begin(9600); // Start Serial communication  
  servoMotor.attach(11); // Connect servo to pin 11  
  pinMode(trigPin, OUTPUT); // Set trig pin as output  
  pinMode(echoPin, INPUT); // Set echo pin as input  
}
```

Libraries: `#include <Servo.h>` is a built-in library that makes servo control easy.

Pin Assignments: Assigns specific pins for hardware components.

Serial Communication: Allows you to print values to a serial monitor for debugging or visualization.

```
long readDistanceCM() {  
  digitalWrite(trigPin, LOW); // Clear trigger  
  delayMicroseconds(2);  
  digitalWrite(trigPin, HIGH); // Send 10us pulse  
  delayMicroseconds(10);  
  digitalWrite(trigPin, LOW);  
  long duration = pulseIn(echoPin, HIGH); // Measure pulse duration  
  return duration * 0.034 / 2; // Convert to cm  
}
```

Pulse Measurement: Uses `pulseIn()` to calculate how long the ultrasonic signal took to return.

Distance Calculation: Converts the time into distance using the speed of sound.

5. Common Arduino Functions in Radar

<u>Function</u>	<u>Purpose</u>
-----------------	----------------

pinMode()	Set pin as INPUT or OUTPUT
digitalWrite()	Send HIGH or LOW signal to pin
digitalRead()	Read digital signal from pin
analogRead()	Read analog value from sensor
delay()	Pause program for given milliseconds
pulseIn()	Measure duration of pulse on a pin
Serial.begin()	Start serial communication
Serial.print()	Send data to serial monitor
servo.attach()	Attach servo motor to pin
servo.write()	Set angle of servo motor

6. The Code Controls the Radar

The code makes the servo sweep the ultrasonic sensor across a 0–[180°](#) range, takes distance readings at each angle, and outputs angle-distance pairs to the serial monitor. This can be visualized as a radar sweep.

8.Working

7.1 METHODOLOGY:

In order to testify the working of this system, after its designing, construction and programming we placed few objects in front of the ultrasonic sensor. As the motor started to rotate, our monitor started to display the output through processing IDE. Hence, when the sensor crossed over the object it showed a red segment with the distance and angle where the object is paced. The first object was placed at the distance of 30.5cm measured through a ruler and the system measured the distance at 32cm. While the second object was placed at a distance of 20 cm and the system measured it as 21cm. Hence the calculated efficiency turned out to be 95%.

System Overview:

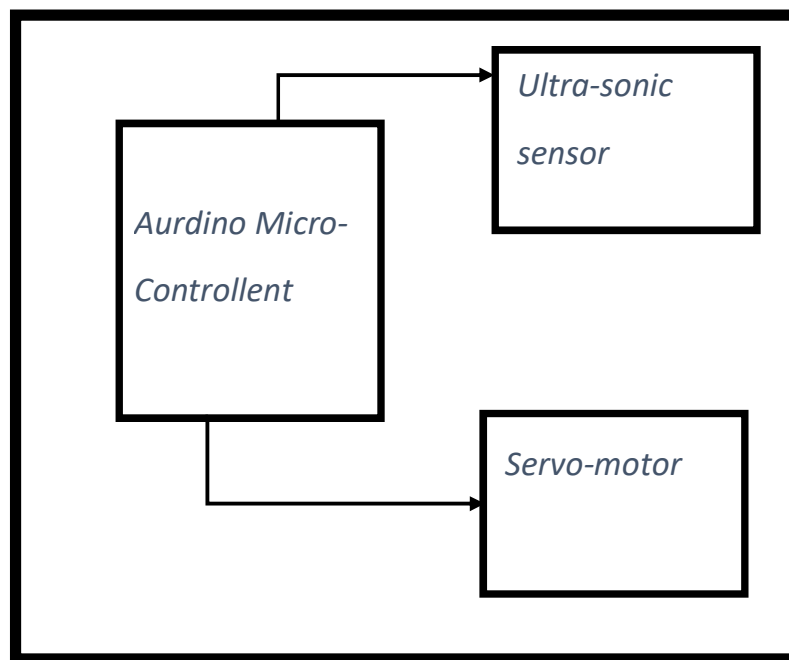


Figure . System hardware description.

The above figure represents a brief overview of this radar system. Here, as it is shown the controller we are using is Arduino, with the input Ultrasonic sensor and the output is the servo motor which rotates 180 degrees. The microcontroller controls all the operations of this system, from rotation of the motors to the obstacle detection of the ultrasonic and representation of the result on the screen.

System block diagram:

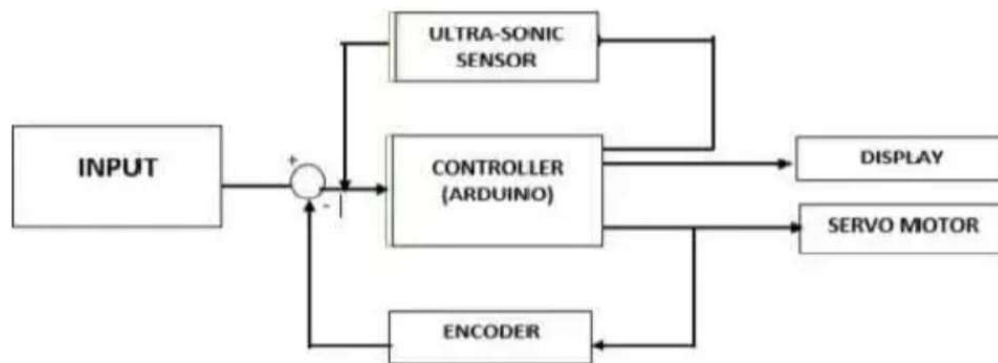
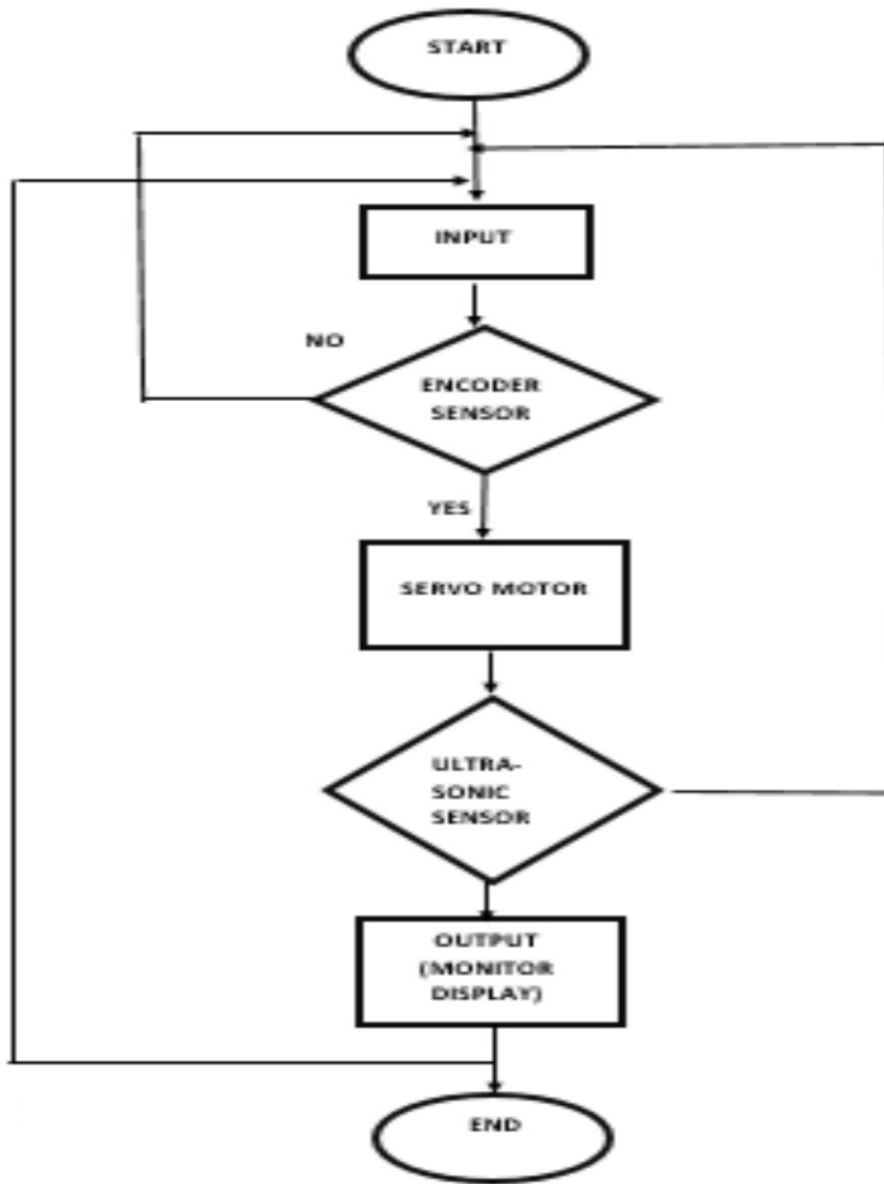


Figure represents the system's block diagram. Here, it can be seen how the work flow in this radar system. The sensor is going to sense the obstacle and determine the angle of incident and its distance from the radar. The servo motor is constantly rotating to and fro, hence making the sensor move. The data obtained is encoded and fed to the processing IDE which represents it on the screen. The results are displayed further in this paper. All these operation are done by Arduino microcontroller from the rotation of the servo, data collection from the sensor, feeding the data to encoder to transferring it to the display.

7.2 Working of the system

The basic objective of our design is to ascertain the distance position and speed of the obstacle set at some distance from the sensor. Ultrasonic sensor sends the ultrasonic wave in various ways by rotating with help of servo motors. This wave goes in air and gets reflected back subsequent to striking some object. This wave is again detected by the sensor and its qualities is analyzed and output is shown in screen indicating parameters, for example, distance and position of object. Arduino IDE is utilized to compose code and transfer coding in Arduino and causes us to detect position or angle of servo motor and it is communicated through the serial port alongside the covered distance of the nearest object in its way. Output of all of this working is shown in the software called processing, it will display the input/output and the range of the object [4]. Implementations of the sensors are done in such a way that ultra-sonic sensor is attached on top of the servo motor because it have to detect the object and its distance. Arduino (micro-controller) will control the ultra-sonic sensor and servo motor and also powered will be given to both of them through micro-controller

Flow chart of Radar System:.



The above flow chart explains the working and the decision flow of this framework. As it can be seen the system starts with an input i.e. when the ultrasonic sensor detects an object, or does not detects any object, at any condition the encoder feeds the information in the controller while the servo keeps constantly rotating. As soon as any obstacle/object is detected by the ultrasonic sensor the data is immediately processed by the controller and is fed to the IDE which shows it on the display screen. Here the process ends with an estimated distance of the object from the system with the angle at which it is placed.

9. APPLICATIONS , Uses & Limitations:

1. Obstacle Detection and Avoidance

Used in: Robotics, drones, autonomous vehicles.

Function: Detects nearby objects to prevent collisions.

Example: An Arduino-based car stops or changes direction when an object is detected within a set range.

2. Security and Surveillance

Used in: Home and perimeter security systems.

Function: Detects motion or intrusions using radar principles (typically ultrasonic or microwave sensors).

Example: An alarm is triggered when motion is detected within a protected area.

3. Speed Detection

Used in: Traffic monitoring, speed measurement experiments.

Function: Calculates speed based on Doppler effect or time-of-flight.

Example: Arduino radar setup to measure the speed of a passing vehicle or object.

4. Level Measurement

Used in: Water tank monitoring, industrial material storage.

Function: Measures the distance from the sensor to the surface of a liquid or solid.

Example: Arduino radar used to determine water level in a tank.

5. Human Presence Detection

Used in: Smart homes, energy-saving systems.

Function: Detects human presence to turn lights or systems on/off.

Example: Arduino radar controls lighting based on occupancy.

6. Gesture Recognition

Used in: Touchless interfaces, assistive technology.

Function: Tracks hand movements and interprets them as commands.

Example: Arduino-based radar detects gestures to control a media player.

7. Application in air force :

It is used in airplanes or aircraft machines which have implemented radar system in it to detect the objects that comes in a way. It is also used to calculate height readings.

8.Application in MARINE:

This radar system also used in ships or marine. It is implemented on big ships to calculate the distance of other boats or ships, with the help of this sea accidents can also be reduced by not colliding. It can also be implemented on ports to see the distance of other ships and to monitor or control the ship movements.

9.Application in METEROLOGY:

Meteorologists also uses radar systems to track or monitor the wind. It has been become an important equipment for climate testing. For example to detect tornados, storms.

8.1:Future Scope

The Arduino-based radar sensor system using COM3 communication port holds significant potential for expansion and practical implementation. Future developments may include:

Integration with IoT and Cloud Platforms

The system can be enhanced to transmit real-time radar data to cloud servers using Wi-Fi or GSM modules. This would allow remote monitoring, data analytics, and predictive maintenance.

AI and Machine Learning

By integrating AI/ML algorithms, the radar system could be trained to classify objects based on size, shape, or motion pattern, enabling advanced object recognition and tracking.

Improved Range and Accuracy

Future versions could use more advanced sensors (like mmWave radar) with higher frequency and better resolution, allowing for longer detection range and more precise distance measurements.

Autonomous Navigation Systems

This radar module can be adapted for use in autonomous vehicles (drones, robots) to detect obstacles and navigate safely in real time.

Multi-Sensor Fusion

Combining radar with other sensors (ultrasonic, LIDAR, GPS) could provide a more robust and reliable sensing system, particularly in harsh or complex environments.

User Interface Enhancements

A more advanced graphical interface could be developed for real-time visualization, including mobile or web dashboards, improving usability and data interpretation.

Security and Surveillance

The radar system can be adapted for perimeter monitoring and intruder detection in security systems, with potential integration into smart surveillance networks.

8.2 Limitations:

1. Limited Range

- * Typically detects objects only within 2 to 4 meters, depending on the ultrasonic or IR sensor used.
- * Not suitable for long-range detection like real military radar systems.

2. Low Resolution and Accuracy

- * Can't distinguish between small or closely spaced objects.
- * The detection angle is usually wide (e.g., $\pm 15^\circ$), causing poor directional accuracy.

3. Limited Speed and Object Tracking

- * Struggles to detect or track fast-moving objects.
- * No Doppler shift capability to measure relative speed.

4. Poor Signal Processing

- * Arduino has limited computational power, restricting advanced signal filtering or data analysis.
- * Real-time processing is basic and prone to noise or false detections.

5. Environmental Sensitivity

- * Performance affected by temperature, humidity, and reflective surfaces.
- * Doesn't work well in noisy or outdoor environments.

6. No Radar Frequency (RF) Use

- * Most Arduino radar setups use ultrasonic or IR sensors, not radio waves, so it's not a true radar.
- * Cannot simulate real radar wave behaviors (e.g., scattering, Doppler, multipath).

7. Limited Interface and Visualization

- * Simple visual displays like 2D plotting in Processing or serial monitors.
- * No support for high-resolution radar imaging or 3D mapping.

8. Not Scalable

- * Cannot be easily scaled to handle multiple targets or integrate into complex systems (e.g., air traffic control, missile guidance)

10. CONCLUSION AND FUTURE ENHANCEMENT

In order to facilitate learning, a number of physical technologies are used in the field of educational technologies, such as computer hardware, software, a graphing calculator, sensor interfaces (LabQuest, Go!Link, Vernier Arduino Interface Shield, etc.), digital cameras, interactive whiteboard tools, and LCD projectors. Additionally, there are also technologies such as blogs, collaborative software, e-portfolios and virtual classes which consist of these technologies used together differently. (Wikipedia. (n.d.); Vernier. (n.d.)).

In a study conducted in recent years, it has been noted that the Arduino has been used primarily in physics education but also in areas such as chemistry, engineering, robotics, electronics and education. It has been stated that the Arduino has made application development possible in various fields from education to engineering. Moreover, it was emphasized that it contributes to students' learning of physics topics, concepts and principles by having them apply basic electrical and electronic knowledge, and to their practical realization of meaningful and permanent learning, through conducting experiments (Duman, 2019, p. 494).

Arduino can be used as an alternative for conducting experiments in physics, taking measurements and collecting data. Therefore, the physics laboratory can be enriched by designing measurement tools and experimental devices with Arduino. There are various studies with an introduction of sensors that can be used with Arduino along with the sizes to be measured. (Fisher & Gould, 2012, p. 8; Organtini, 2016, p. 45-46).

Likewise, there are studies related to the reorganization of electrical experiments carried out in physics laboratories with Arduino (Kirikkaya & Basaran, 2017, p. 351). In this context, it has been noted that projects could be developed to determine the electrical, mechanical and magnetic properties of substances (Bouquet, Bobroff, Fuchs-Gallezot, & Maurines, 2017, p. 3; Huang, 2015, p. 26.1205.2).

In relation to the use of Arduino in physics education; temperature dependency of electrical resistance (Sari & Kirindi, 2019, p. 688), investigation of simple harmonic motions within the scope of kinematic measurements (Music, 2017, p. 1; Tong-on, Saphet & Thepnurat, 2017, p. 1), have shown in various studies that magnetic field measurements can be performed (Organtini, 2016, p. 57).

In one of his studies, Nichols contributed to the development of measurement tools that can be used in the laboratory by showing that the data received from Arduino Uno could be transferred to programs such as Excel, LabVIEW, MATLAB (Nichols, 2017, p. 226).

In another study, a free fall experiment was examined by designing an Arduino-based experiment. This study showed that the gravitational acceleration of an object released from a certain height could be calculated using location and time data with the HC-SR04 ultrasonic distance sensor and Arduino (Moya, 2018, p. 1).

In this study, real-time monitoring of the values for physical quantities is provided with the voltage, distance and time interval measurements performed. The variety of sensors (acceleration, temperature, light, sound, magnetic field sensors, etc.) that can be used on the Arduino platform can provide many and interesting experiments in the fields of Mechanics, Thermodynamics, Electromagnetism and Optics (Bouquet et al., 2017, p. 2; Oprea, 2018, p. 99).

As a result, due to the inexpensiveness of Arduino and its sensors, the variety of sensors to be used in physics experiments, and the ease of accessing various resources on this subject from

the internet, it is thought that the use of Arduino in physics course laboratory activities will be beneficial to teachers, students and those who work as amateurs or professionals on this subject. In addition, since it is possible to design many measuring devices that can be used in physics laboratories with Arduino, it can be made easier and more enjoyable to learn physics in depth with these measuring tools.

REFERENCES

- <https://www.arduino.cc>
- <https://processing.org>
- “Arduino Cookbook” by Michael Marglois
- “Practical Electronics for Inventors” by Paul Scherz & Simon Monk

Learning Outcomes:

1. Understanding of Radar Technology:

Gained knowledge about radar principles including detection, ranging, and signal reflection.
Explored the historical evolution and modern applications of radar systems.

2. Practical Implementation Using Arduino:

Learned how to interface an ultrasonic sensor and servo motor with the Arduino UNO microcontroller.

Understood how Arduino can be used for prototyping electronic systems.

3. Hardware & Software Integration:

Understood the operation and use of key hardware components: ultrasonic sensor (HC-SR04), servo motor, Arduino UNO, jumper wires, and breadboard.

Developed skills in using the Arduino IDE and Processing IDE for sensor data visualization.

4. Programming Skills:

Acquired coding experience in Arduino C/C++.

Developed functions to control hardware (e.g., servo movement, ultrasonic measurements).

Used serial communication to interface hardware and software.

5. Object Detection & Measurement:

Measured distance and angle of obstacles using ultrasonic reflection principles.

Learned the concept of non-contact distance measurement and basic object tracking.

6. Design and Testing:

Designed a working prototype capable of scanning an area and detecting obstacles.

Verified sensor accuracy and calculated system efficiency (approx. 95%).

7. Visualization and Analysis:

Created a radar-like graphical interface using Processing to display real-time data.

8. Applications and Future Prospects:

Identified practical applications in robotics, surveillance, autonomous vehicles, and education.

Understood limitations and proposed enhancements (e.g., IoT integration, AI/ML, better sensors).

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any work would be incomplete without the expression of gratitude to the people who made it possible. I acknowledge all those guidance and encouragement that has made me to do what has been done so far.

First and foremost, I would like to thank my project guide, Dr. Asha Kiran Pakkala, Assistant Professor and head of the Department of Physics for her encouragement, teaching, help and support. I take this opportunity to convey my heartfelt thanks to her.

I am thankful to Mrs. Sukhalatha, Assistant Professor, the Department of Physics for providing the necessary facilities during the course of the project work.

I wish to thank all my friends for their co-operation, help and support. I am thankful to all the non-teaching staff for their timely help.

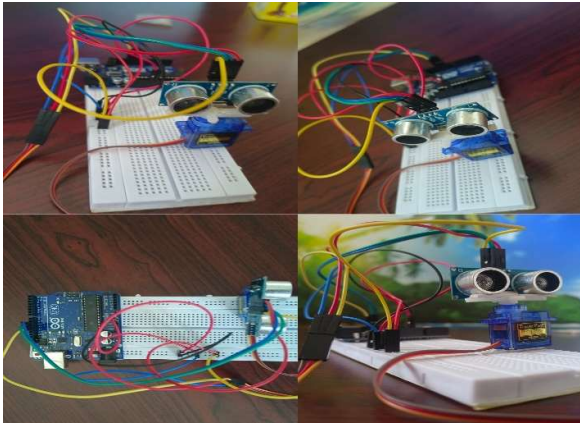
Finally, I thank all those who helped me directly/indirectly and extended their support in the completion of my project work.

(SHREYASK)

Place: Canara college, Mangalore

Date:

APPENDIX



Fig(1):The Aurdino with radar circuit



Fig(2):The Aurdino without detection



Fig(3):The Aurdino with detection